WEEK 11 - PYTHON PROGRAMMING

Writing the Project Documents

Requirement, Feature, Function, UML: four views of one program.

DOC 1 · REQUIREMENT
what must hold

DOC 2 · FEATURE
what the user sees

DOC 3 · FUNCTION
what code promises

DOC 4 · UML
how it connects

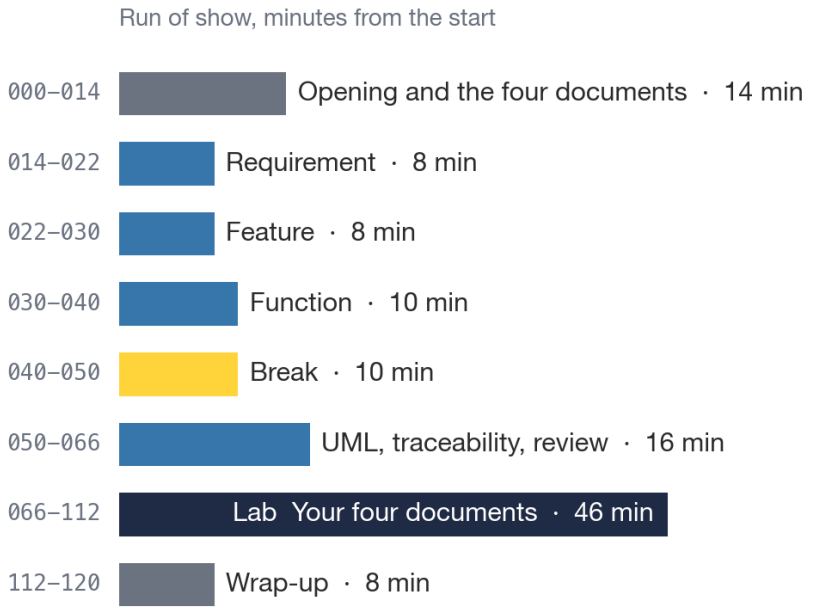
120 minutes, including a break

Instructor: MinGi KYUNG

Project Guidance and Document Scope

What the project is, and what these four documents are for.

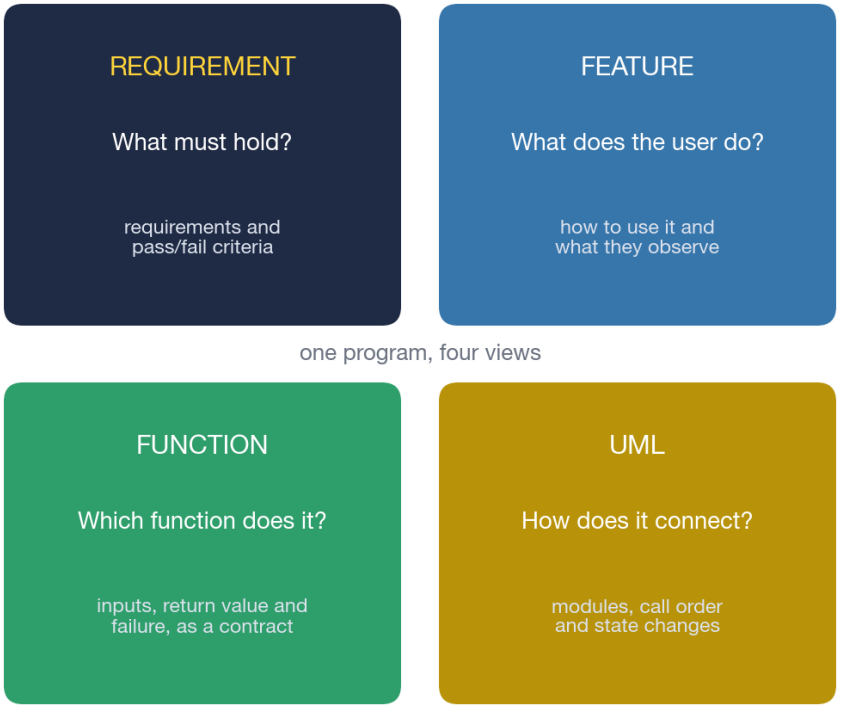
- Python 3 and the standard library. Data kept in a file, reported on the screen.
- Shared checks: bad input, a missing file, the file unchanged after a failure.
- Design: the problem, the feature scope, the module structure and the data format.
- The four documents are a guide. Submission items and weights follow the syllabus.



The Role of the Four Documents

Four questions about the same program.
Each document answers one of them.

- Requirement: what must hold? The requirements, and a pass/fail criterion for each.
- Feature: what does the user do? How to use it, and what they can observe.
- Function: which Python function does it? Inputs, return value and failure, as a contract.
- UML: how does it connect? Module relations, the order of calls, and state changes.

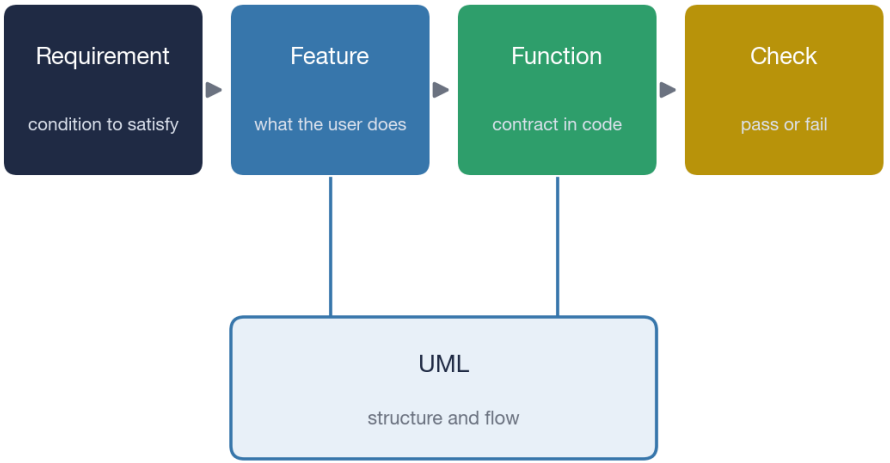


Linking Requirements to Implementation

The same project, seen from the purpose, the user, the code, and the checks.

- A requirement is a condition to satisfy. A feature is what the user does to satisfy it.
- A function is the code's promise. A check decides whether the promise holds.
- UML sits under feature and function: the structure and the flow that connect them.

> **RULE** The same ID in every document: a requirement traces to its code and its check.



the same ID in every document makes a requirement traceable

Shared Example: The Study Logger

One example for every document today: the study logger from weeks 9 and 10.

- Basic features: log one entry, reload the file, show weekly totals per course.
- Result report: one line per course, largest first, or "Not saved" with the reason.
- Duties: refuse bad input; file unchanged on failure; survive a missing file.

> **TIP** Minutes are what the student types. Timing with a clock is an extension feature.

OUTPUT

```
$ python3 main.py
Date (YYYY-MM-DD): 2026-11-10
Course: python
Minutes: 90
Saved.
python   180 min
stats     45 min
```

Requirement: The Specification

DOC 1 · REQUIREMENT

What must hold, for whom, and how far.
Each line can be judged pass or fail.

- Purpose and scope: who has the problem, what the program solves, and where it stops.
- Requirement items: an ID, the statement, a priority (Must, Should, Could) and a check.
- Functional: log an entry, apply the input rules, report totals, keep the data.
- Constraints: standard library only, input checked, file unchanged on failure.

OUTPUT

```
ID      statement              prio  check
REQ-01  Save an entry with     Must  one new row
        date, course and             in study.csv
        positive minutes
one line, one condition; the check is
something a classmate can do at the keyboard
```

Requirement Example

DOC 1 · REQUIREMENT

Every requirement carries an observable check. No check, no requirement.

- The statement says what, not how. No function names, no file formats.
- The check is something a classmate can do at the keyboard in a minute.
- Must requirements are the MVP. Should and Could wait until the MVP runs.

PAYOFF These checks become test_logic.py next session and the test list in week 12.

OUTPUT

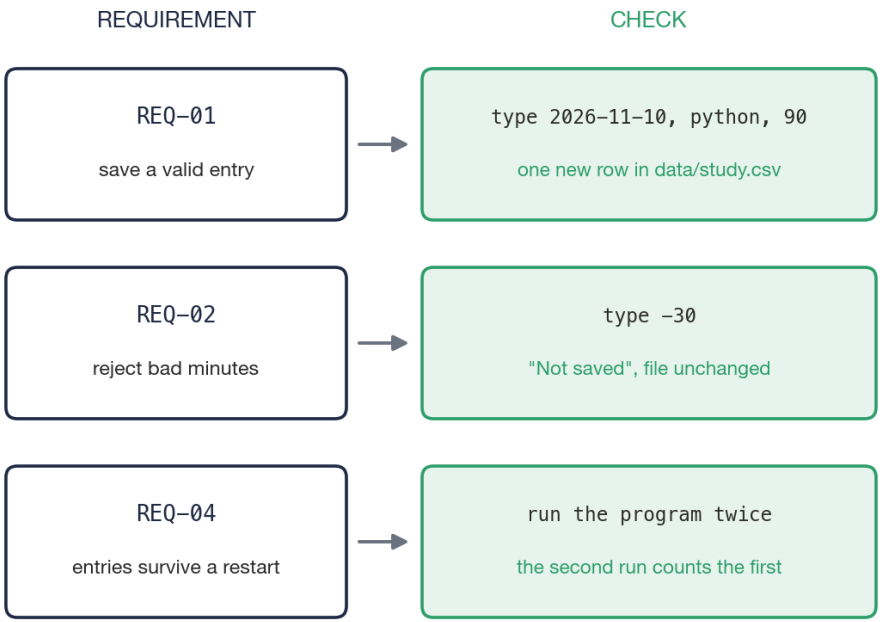
```
REQ-01  Save an entry: date, course,
        positive minutes              Must
        check: 2026-11-10, python, 90
               -> one new row in study.csv
REQ-02  Reject minutes that are not a
        positive whole number         Must
        check: -30 -> Not saved, file unchanged
REQ-03  Weekly totals per course,
        largest first                 Must
        check: python 90 + 90, stats 45
               -> python 180 printed first
REQ-04  Keep entries between runs     Must
        check: run twice; the second
               run counts the first
```

Matching Requirements to Checks

DOC 1 · REQUIREMENT

Each requirement sentence gets an input condition and a result you can observe.

- Input condition: exactly what is typed, or what the file holds before the run.
- Observed result: what is printed, and what the file contains afterwards.
- A check that needs its author to explain it is not a check yet.



input condition on top, observed result below

Feature: The User-Facing Specification

DOC 2 · FEATURE

The program as the user meets it: a command, prompts, and what comes back.

- Identification: feature ID, name, purpose, basic or extension.
- How to use: the command, the prompts, the allowed values, the preconditions.
- Behavior and result: the normal flow, and the output the user checks.
- Exceptions and links: bad input, failures, and the requirement IDs this feature serves.

OUTPUT

```
FEAT-xx  <name>              basic | extension
purpose  one sentence
usage    command, prompts, allowed values
flow     step 1 ... step n, the normal path
result   what the user sees on success
errors   each failure and its message
serves   REQ-xx, REQ-yy
```

Feature Example: Logging One Entry

DOC 2 · FEATURE

One feature sheet, filled in for F1 of the study logger.

- The usage line is what the demo will type. Write it so that a stranger could.
- Errors are listed one by one, each with its message. "Handles errors" is not a line.
- The serves line is the traceability link back to the requirements.

OUTPUT

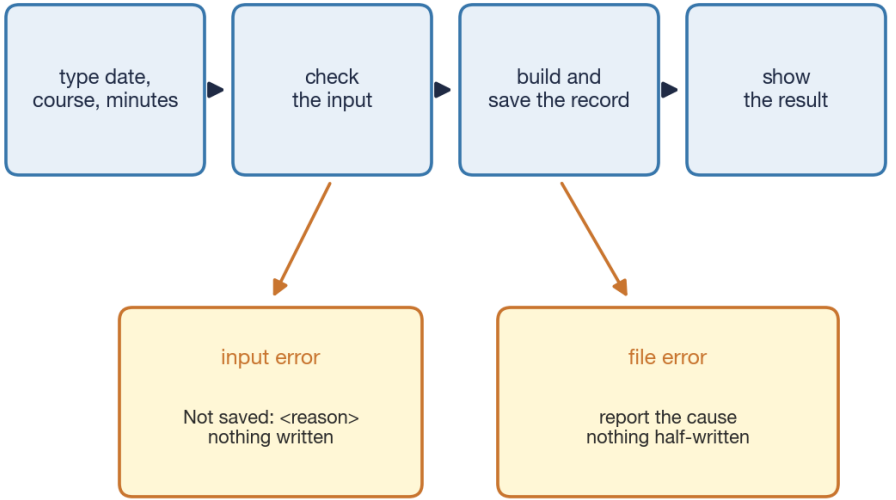
```
FEAT-01  Log one entry          basic
purpose  record one study session
usage    python3 main.py, then three prompts:
         date YYYY-MM-DD, course, minutes > 0
flow     check input -> build record
         -> append one row to data/study.csv
result   "Saved."
errors   bad minutes or empty course:
         "Not saved: <reason>", file unchanged
         data/ missing: report it, write nothing
serves   REQ-01, REQ-02, REQ-04
```

A Feature's Normal Path and Error Path

DOC 2 · FEATURE

Define what the user sees on the good day and on the bad day. Both belong in the sheet.

- Normal path: four steps, each with something the user can observe.
- Input error: a sentence with the reason, and nothing written to the file.
- File error: the cause reported, nothing half-written, the program ends cleanly.



write the normal result and the failure results together

Function: Python Function Design

One Python function, one contract. Your week 10 design document has the fields.

- Signature: name, parameters with types, return type.
- Inputs and results: allowed values and units, what success returns, how failure shows.
- Where it runs: its module, its caller, keyboard or file access or neither.
- Failure and resources: raise or return None; who opens and closes the file.

OUTPUT

```
make_record(date: str, course: str,
            minutes_text: str) -> dict
module   logic.py        called by  main()
I/O      none            pure       yes
returns  {"date", "course", "minutes": int}
fails    ValueError: empty course, or
         minutes not a whole number above 0
```

Function Example: make_record()

The contract in the docstring, the body under it, and one call that shows the promise kept.

- Input: three strings from input(). The course is trimmed and lower-cased here only.
- Called by main() before anything is written. Returns one dict, minutes as int.
- Failure: raises ValueError with a reason. main prints Not saved and writes nothing.

PYTHON

```
def make_record(date, course, minutes_text):
    """Return {"date": str, "course": str,
    "minutes": int}. The course is trimmed and
    lower-cased. Raises ValueError if course is
    empty or minutes_text is not a whole number
    above 0.
    """
    course = course.strip().lower()
    if not course:
        raise ValueError("course must not be empty")
    minutes = int(minutes_text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return {"date": date.strip(), "course": course,
            "minutes": minutes}

rec = make_record("2026-11-10", " Python ", "90")
print(rec["course"], repr(rec["minutes"]))
```

OUTPUT

```
python 90
```

Decomposing into Functions

Six functions, six single responsibilities. The verb test from week 10 names them.

- Two rule functions, two file functions, one report function, and main() that calls them.
- Each name is a verb plus a noun. Each line here is one row of your contract table.
- If a function needs "and" to describe it, split it before you write the contract.

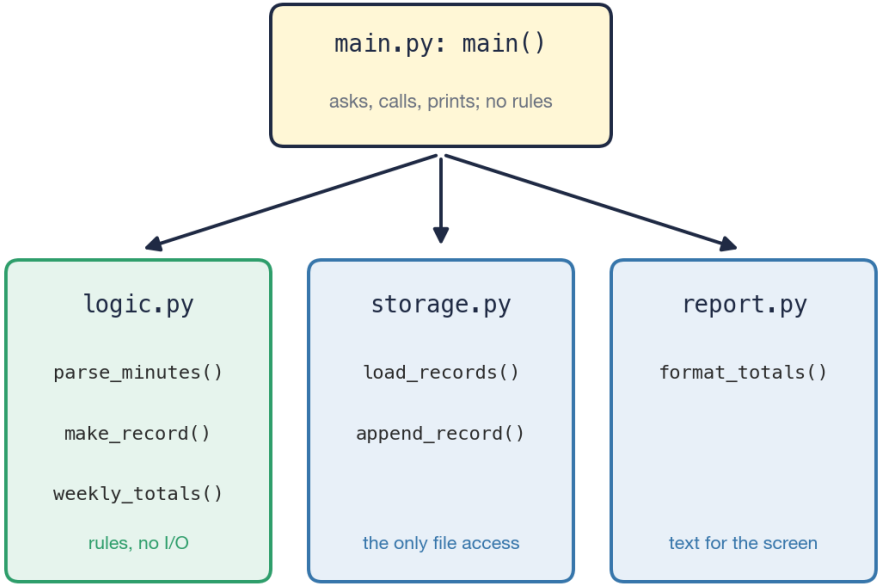
OUTPUT

```
parse_minutes()   minutes text -> positive int,
                  or ValueError
make_record()     date, course, minutes
                  -> one record dict
append_record()   one record -> one row in
                  data/study.csv
load_records()    the file -> list of records,
                  [] if the file is missing
weekly_totals()   records -> {course: minutes}
format_totals()   totals -> lines, largest first
```

Functions and Modules: Who Does What

After main() asks, each module answers one kind of question. None of them asks the user anything.

- main.py asks, calls and prints. It holds no rules.
- logic.py holds the rules and never touches the keyboard or the disk.
- storage.py is the only module that opens data/study.csv. report.py turns results into lines.



main asks; each module answers one kind of question

BREAK

Halftime

Ten minutes. Three documents down, one to go.

- After the break: UML, the diagram that connects the other three, and the traceability chain.
- Then the lab: your own four documents, for your own project.

UML: Choosing a Diagram by Purpose

DOC 4 · UML

You do not need every diagram. Pick the one that answers your reader's question.

- Use case: who uses the program, and for which goals.
- Module (component): which file imports which, with arrows that point one way.
- Sequence: the order of calls between main() and the modules, for one run.
- State or activity: how one entry changes state, or the steps with their error branches.

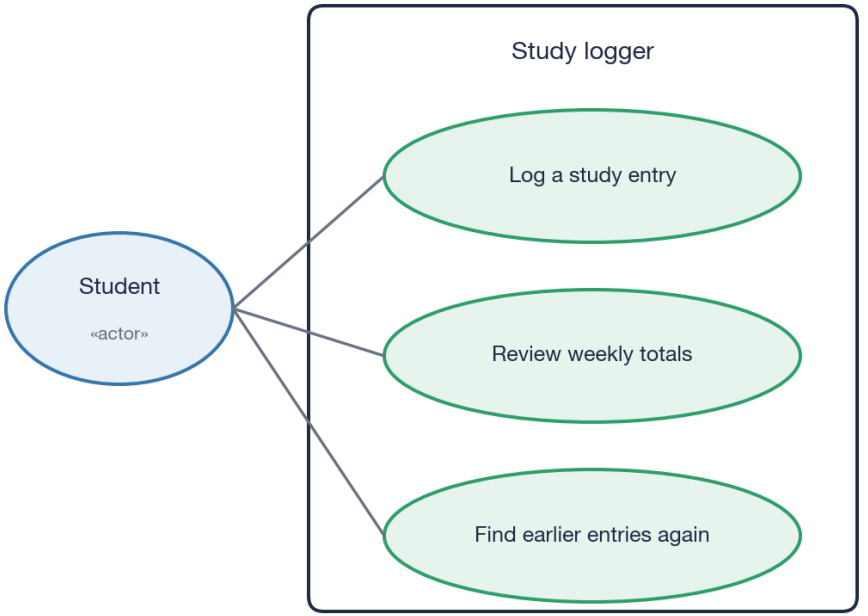
OUTPUT

```
"Who uses it, and for what?"       use case
"Which file imports which?"        module diagram
"In what order are calls made?"    sequence
"What happens to one entry?"       state
```

UML Use Case Example

The user and the goals the system offers. Goals, not functions.

- One actor: the student. The system boundary is the study logger.
- Three goals: log an entry, review weekly totals, find earlier entries again.
- Function names do not appear here. The order of calls belongs in the sequence diagram.

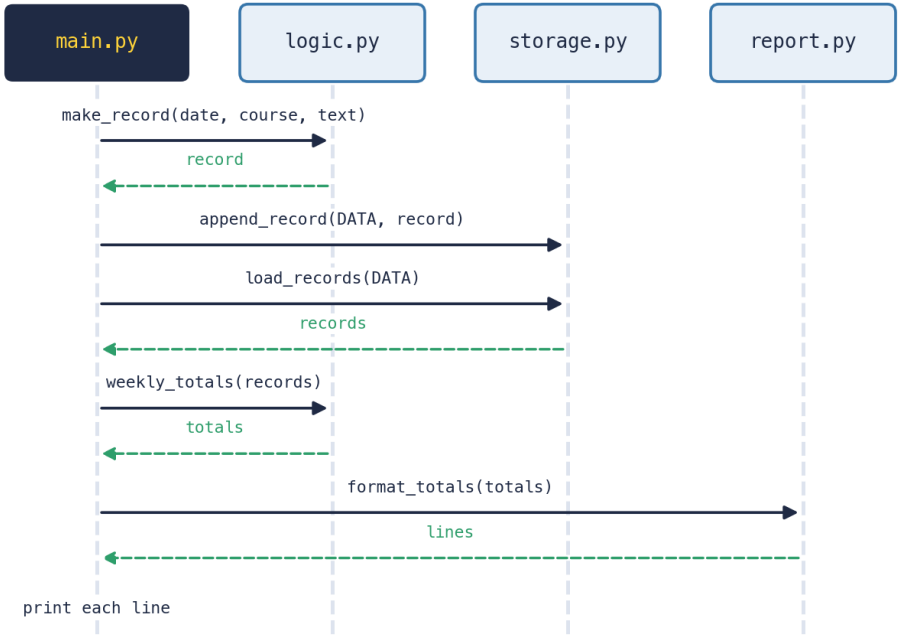


goals, not functions: the call order is the sequence diagram

UML Sequence Example: One Run

One run, normal path, time flowing downward. Every arrow is a call from your contract table.

- main() calls logic first, so nothing is written before the record is valid.
- Each return arrow carries one data shape: a record, a list of records, a totals dict, lines.
- The failure path, a ValueError, is a separate branch or a second, shorter diagram.

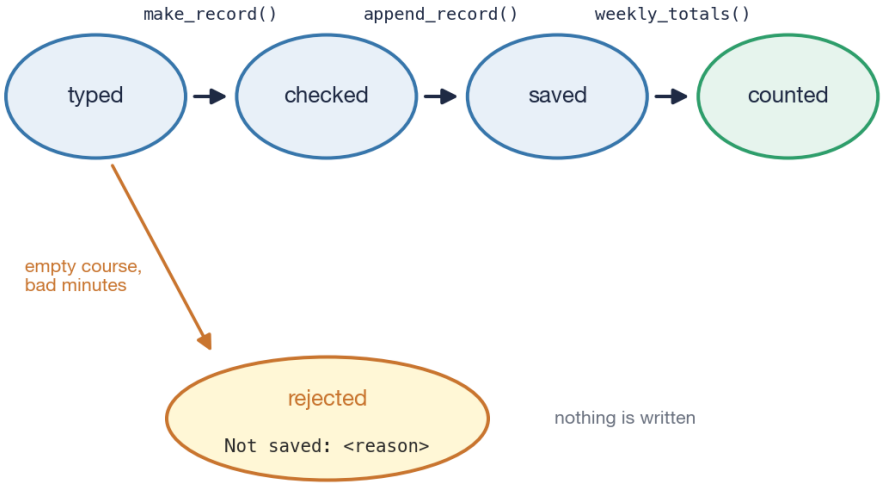


normal path; time flows downward; a ValueError is a separate branch

UML State Example: One Entry

Four states and one dead end. Each transition is a function from the table.

- typed, then checked by make_record(), saved by append_record(), counted by weekly_totals().
- rejected: an empty course or bad minutes ends the entry with "Not saved". Nothing is written.
- A file error after the check is also reported, and nothing is half-written.



Traceability Across Documents

DOC 4 · UML

Write the requirement ID in the feature, the function, the diagram and the test. Then nothing is lost.

- REQ-02 says what must hold: minutes that are not positive are rejected.
- FEAT-01 shows it to the user, parse_minutes() enforces it, the state diagram draws it, TEST-02 proves it.
- A grader, or you in week 14, can follow the chain in either direction.



one ID, five documents, readable in either direction

Document Review and Demo Preparation

DOC 4 · UML

Five questions before you call the documents done.

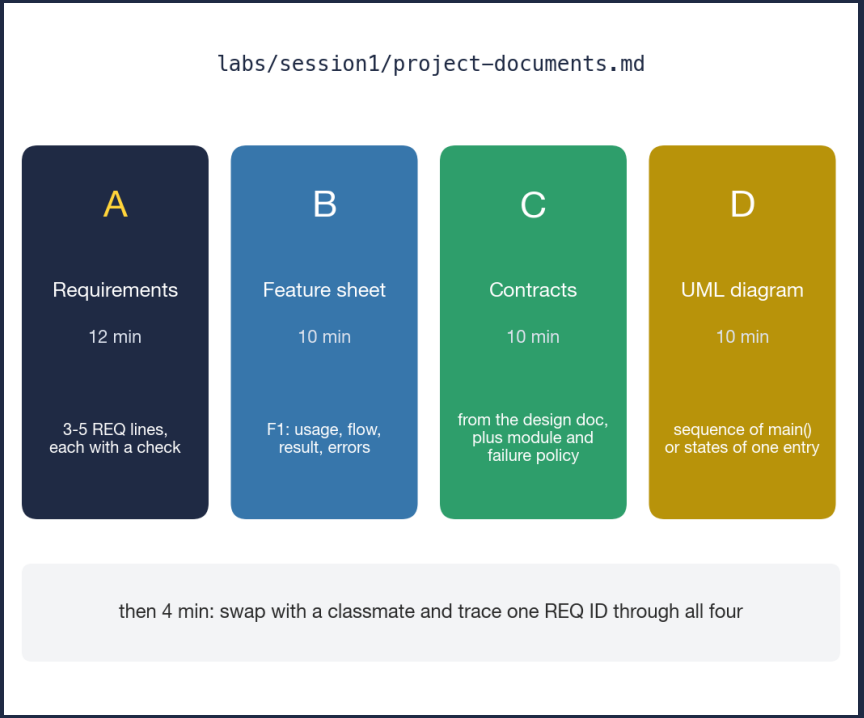
- Scope: are basic and extension features separated, and is the MVP line visible?
- Completion: can each requirement be judged pass or fail with its check?
- Consistency: do the feature sheet, the function contracts and the UML flow agree?
- Data checks: are bad input, a missing file, the header row, and "file unchanged on failure" covered?
- Reproduction: are the setup, the run command, one normal scenario and one error scenario written down?

LAB

Lab: Your Four Documents

46 minutes. Four short documents for your own project, with one ID scheme.

- A Requirements, 12 min. B Feature sheet for F1, 10 min.
- C Function contracts, 10 min. D One UML diagram, 10 min.
- Then 4 minutes: swap with a classmate and trace one REQ ID through all four.



Submit and Next Session

Four documents, one ID scheme. Next session the code starts to run against them.

- Submit labs/session1/project-documents.md via the LMS today.
- Keep your REQ and FEAT IDs. They go into commit messages, test names and the README.
- Next session: From Design to Running Code. F1 runs end to end on a feature branch.
- Bring a laptop where your repository is cloned, git status is clean, and python3 main.py runs.

> **RULE** Must scope stays frozen. A new idea becomes a Should or Could line, not a new Must requirement.